



Bedienungsanleitung



Bedienungsanleitung – RAT-Backend

Diese Anleitung zeigt, wie die API gestartet wird und wie man sie mit Beispiel-Requests benutzt.



1. API starten

Voraussetzungen

Python 3 muss installiert und auf dem PATH sein.

Internet beim ersten Start (die Abhängigkeiten werden automatisch installiert).

Schnellstart (mit den Run-Skripten)

Die Run-Skripte im Projekt-Root legen automatisch eine virtuelle Umgebung (`.venv`) an, installieren `src/requirements.txt` und starten den Server.

Windows (in einem Schritt):

```
git clone https://github.com/sumpfel/RAT-Backend.git &&  
cd RAT-Backend && set HOST=0.0.0.0 && set  
PORT=8080 && run.bat
```

Linux / macOS:

```
git clone https://github.com/sumpfel/RAT-Backend.git &&  
cd RAT-Backend  
  
./run.sh # http://127.0.0.1:8000  
HOST=0.0.0.0 PORT=8080 ./run.sh # auf allen Interfaces,  
Port 8080  
RELOAD=1 ./run.sh # Auto-Reload  
(Entwicklung)
```

Windows (PowerShell):

```
./run.ps1 # http://127.0.0.1:8000
./run.ps1 -BindHost 0.0.0.0 -Port 8080
./run.ps1 -Reload
```

Nach dem Start:

API-Basis: **http://127.0.0.1:8000** (bzw. der gewählte Host/Port)

Swagger UI (interaktive Doku): **http://127.0.0.1:8000/docs**

Logs: Konsole **und** Datei `src/api.log`

“

Standard-Admin: Bei einer frischen Datenbank wird automatisch ein Admin-Account `admin` / `admin` angelegt. **Bitte das Passwort nach dem ersten Login ändern.**

Direkt mit uvicorn starten (ohne Run-Skript)

Vom **Projekt-Root** aus:

```
uvicorn src.main:app --host 127.0.0.1 --port 8000
```

(Voraussetzung: die Abhängigkeiten aus `requirements.txt` sind installiert.)

Datenbank initialisieren / Testdaten anlegen (optional)

`init_db.py` legt die Tabellen an und füllt eine **frische** DB mit Dummy-Testdaten (zwei User `admin` / `bob`, zwei Geräte, Interfaces, eine Verbindung, Berechtigungen, ein Login und SNMP-Settings). Vom Projekt-Root:

```
python init_db.py
```

Es schreibt direkt in `src/RATBASE.db`. Ein erneuter Lauf auf einer bereits gefüllten DB überspringt das Seeding (DB-Datei löschen, um neu zu seeden). Der Server selbst initialisiert die DB beim Start ebenfalls — `init_db.py` ist also optional.

Cloud-Datenbank (optional)

Standardmäßig wird eine lokale SQLite-Datei genutzt. Für eine Cloud-DB (Supabase / Railway / PlanetScale) `src/.env.example` nach `src/.env` kopieren und `DATABASE_URL` setzen (siehe Kommentare in der Datei), z. B.:

```
DATABASE_URL=postgresql+psycopg2://  
USER:PASSWORD@HOST:5432/DBNAME
```

Den passenden Treiber installieren (`pip install psycopg2-binary` bzw. `pymysql`).



2. Beispiel-Requests

Die Beispiele nutzen `curl`. Basis-URL hier: `http://127.0.0.1:8000`. Fast alle Endpunkte brauchen einen **JWT** im Header `Authorization: Bearer <TOKEN>`.

2.1 Login (JWT holen)

Login läuft über ein OAuth2-Formular (`application/x-www-form-urlencoded`):

```
curl -X POST http://127.0.0.1:8000/user/login \  
-d "username=admin&password=admin"
```

Antwort (`200`):



```
{ "access_token": "eyJhbGc...", "token_type": "bearer" }
```

Token in eine Variable legen (Linux/macOS):

```
TOKEN=$(curl -s -X POST http://127.0.0.1:8000/user/login  
-d "username=admin&password=admin" | python3 -c  
"import sys,json;print(json.load(sys.stdin)  
['access_token'])")
```

2.2 Eigene Daten / alle User

```
curl http://127.0.0.1:8000/user/me -H  
"Authorization: Bearer $TOKEN"  
curl http://127.0.0.1:8000/user/ -H  
"Authorization: Bearer $TOKEN"
```

2.3 User anlegen (Passwort-Policy: ≥ 8 Zeichen, Buchstabe + Ziffer)

```
curl -X POST http://127.0.0.1:8000/user/register \  
-H "Content-Type: application/json" \  
-d '{"username":"bob","password":"bobbob12","is_admin":false,"canCreate":
```

Doppelter Username → 409

Zu schwaches Passwort → 400

2.4 NetworkObject anlegen / auflisten / bearbeiten / löschen

```
# anlegen (201)  
curl -X POST http://127.0.0.1:8000/networkObject/ \  
-H "Authorization: Bearer $TOKEN" -H "Content-Type:  
application/json" \  
-d '{"name":"PC-1","type":"PC","x":10,"y":20,"os":"Linux","cpu":"i5","gpu"
```

```
# auflisten (200)
curl http://127.0.0.1:8000/networkObject/ -H
    "Authorization: Bearer $TOKEN"

# bearbeiten (200)
curl -X PUT http://127.0.0.1:8000/networkObject/1 \
    -H "Authorization: Bearer $TOKEN" -H "Content-Type:
    application/json" \
    -d '{"name":"PC-1","type":"PC","x":30,"y":40,"os":"Linux","cpu":"i7","gpu":'

# löschen (200)
curl -X DELETE http://127.0.0.1:8000/networkObject/1 -H
    "Authorization: Bearer $TOKEN"
```

Doppelter Name beim Anlegen → 409, unbekannte ID → 404.

2.5 Filterung, Sortierung & Pagination (Query-Parameter)

Alle Parameter sind optional:

```
# Namens-Suche + nur Typ "PC", nach Name sortiert, erste
    10
curl "http://127.0.0.1:8000/networkObject/?
    name=pc&type=PC&sort_by=name&order=asc&limit=10&offset=0"
    -H "Authorization: Bearer $TOKEN"

# Interfaces eines Geräts, nur "up"
curl "http://127.0.0.1:8000/networkObjectInterface/?
    network_object_id=1&is_up=true&limit=50" \
    -H "Authorization: Bearer $TOKEN"

# Connections ab einer Mindest-Geschwindigkeit, nach speed
    absteigend
curl "http://127.0.0.1:8000/networkObjectConnection/?
    min_speed=1000&sort_by=speed&order=desc" \
    -H "Authorization: Bearer $TOKEN"
```

2.6 Statistik / Aggregation

```
curl http://127.0.0.1:8000/statistics/  
summary -H "Authorization: Bearer  
$TOKEN"  
curl http://127.0.0.1:8000/statistics/objects-by-  
type -H "Authorization: Bearer $TOKEN"  
curl http://127.0.0.1:8000/statistics/connections-by-  
type -H "Authorization: Bearer $TOKEN"  
curl http://127.0.0.1:8000/statistics/interfaces-per-  
object -H "Authorization: Bearer $TOKEN"
```

Beispiel `objects-by-type` (GROUP BY `type` + COUNT):

```
[ { "type": "PC", "count": 4 }, { "type": "Switch",  
  "count": 2 } ]
```

2.7 Berechtigung vergeben (User darf ein Objekt sehen/bearbeiten)

```
# level: 0=Hidden 1=See 2=Edit 3=Admin 4=Owner  
curl -X POST http://127.0.0.1:8000/  
networkObjectPermission/ \  
-H "Authorization: Bearer $TOKEN" -H "Content-Type:  
application/json" \  
-d '{"network_object_id":1,"permissions":2,"target_user_id":2}'
```



3. HTTP-Statuscodes (Übersicht)

Code	Bedeutung in dieser API
200	erfolgreiches GET / PUT / DELETE
201	erfolgreiches Anlegen (POST)
400	

Code	Bedeutung in dieser API
	Validierungsfehler / Passwort-Policy / ungültige Berechtigungsänderung
401	falsche Login-Daten oder ungültiges/abgelaufenes Token
403	angemeldet, aber zu wenig Rechte für die Aktion
404	unbekannte ID (oder ein für dich verborgenes Objekt)
409	Konflikt: Username oder NetworkObject-Name existiert bereits

